

Weekend Take-Home Task

Metadata-Driven Full-Stack Application Builder

Full-stack architect | Domain agnostic | Weekend take-home | Candidate may choose the technology stack

Objective

Build a domain-agnostic, metadata-driven full-stack application that lets users define metadata and render dynamic forms, widgets, tables, charts, and dashboards from configuration rather than hard-coded UI. The goal is not to build a full no-code product; focus on a working demonstration with clear separation between metadata, rendering, persistence, and UI components.

1. Admin / Metadata Configuration Area

- Create and edit basic metadata through the UI.
- Support form definitions, field definitions, labels, field types, required flags, and basic layout sections.
- Advanced metadata may be seeded through JSON, database seed data, or configuration files.

2. Runtime Application Area

- Read metadata and dynamically render forms, validation, conditional field visibility, records, tables, widgets, charts, and dashboards.
- Support multiple form and dashboard definitions.
- No authentication is required.

Functional Requirements

- Persist both metadata/configuration and submitted user data.
- Render dynamic forms based on metadata.
- Support field types: text, number, date, select/dropdown, checkbox or toggle.
- Support validation rules from metadata, including required fields.
- Support conditional visibility, such as showing or hiding fields based on another field value.
- Display submitted data in a table.
- Render at least one KPI/stat widget and at least one chart from submitted data.
- Render at least one dashboard from metadata.
- Run locally using clear setup instructions.

Suggested Metadata Concepts

You are free to design the schema, but your solution should consider concepts such as application definition, form definition, field definition, validation rule, layout/section definition, widget definition, dashboard definition, submitted record, and metadata version if implemented as a bonus.

Technical Constraints

There are no prescribed technology constraints. Choose the frontend, backend, database, libraries, and tooling you think are appropriate. Third-party libraries and AI tools are allowed, but disclose major tools used. Do not simply wrap an off-the-shelf no-code or form-builder product.

Mandatory Requirements

- UI must be rendered from metadata/configuration.
- Include a basic admin UI for creating or editing metadata.
- Persist metadata and submitted user data.
- Include dynamic forms and dashboards.
- Include at least one table, one KPI/stat widget, and one chart.
- Support validation and conditional field visibility.
- Include a GitHub repository, screenshots, and clear local setup instructions.

Deliverables

- GitHub repository.
- README with setup instructions, local run steps, architecture overview, metadata/schema explanation, assumptions, and trade-offs.
- Screenshots showing admin/config UI, rendered form, submitted data table, and dashboard with KPI/chart widgets.
- Lightweight tests covering metadata-to-UI rendering, validation logic, and one backend save/read flow.
- Deployment is optional and counts as a bonus.

Bonus Points

- Metadata versioning; computed fields; cross-field validation.
- Plugin/widget registry architecture; JSON import/export for metadata.
- Dashboard layout editor or drag-and-drop builder.
- Audit log; Docker setup; deployed demo.
- Storybook/component documentation; responsive design.
- Clean separation between metadata, renderer, persistence, and UI components.